LẬP TRÌNH PYTHON CHO MACHINE LEARNING

Từ Python cơ bản đến Machine Learning

Ngô Hoàng Dũng - Lê Hoàng Ân - Nguyễn Thị Hà - Lê Ngọc Ánh Dương

Ngày 27 tháng 10 năm 2026

Mục lục

1 Giới thiệu về Python **1**
1.1 Python là gì? . . . 1
1.2 Lịch sử phát triển của Python . . . 1
1.2.1 Khởi nguồn & Phiên bản thử nghiệm (1989 – 1991) . . . 1
1.2.2 Python 1.0 & Chuẩn hóa quản trị (1994 – 1999) . . . 1
1.2.3 Python 2.0 & Sự trỗi dậy của Cộng đồng (Tháng 10/2000) . . . 2
1.2.4 Python 3.0 – ”Cuộc đại tái thiết” (Tháng 12/2008) . . . 2
1.2.5 Kỷ nguyên AI/Machine Learning & Khai tử Python 2 (2015 – Nay) . . . 2
1.3 Guido van Rossum – tác giả của Python . . . 2
1.4 Tên gọi Python . . . 2
1.5 Hướng dẫn cài đặt Python và Anaconda . . . 2
1.5.1 Cài đặt Python . . . 2
1.5.2 Cài đặt Anaconda/Miniconda . . . 3
1.5.3 Tạo môi trường Conda . . . 4
1.6 Tóm tắt chương . . . 4

2 Cú pháp Python cơ bản **5**
2.1 Biến và Kiểu dữ liệu . . . 5
2.1.1 Các kiểu dữ liệu cơ bản . . . 5
2.2 Toán tử trong Python . . . 6
2.2.1 Toán tử cơ bản . . . 6
2.2.2 Toán tử gán mở rộng . . . 6
2.3 Hàm toán học trong Python . . . 6
2.3.1 Các hằng số toán học . . . 7
2.3.2 Các hàm toán học thường dùng . . . 7
2.4 Cấu trúc dữ liệu nâng cao . . . 7
2.4.1 List (Danh sách) . . . 7
2.5 Cấu trúc điều khiển và Vòng lặp . . . 7
2.5.1 Câu lệnh điều kiện if-else . . . 7
2.6 Tóm tắt chương . . . 8

3 Lập trình hướng đối tượng trong Python **9**
3.1 Lớp và Đối tượng (Class and Object) . . . 9
3.2 Các phương thức đặc biệt (Magic Methods) . . . 9
3.2.1 Phương thức `__init__` và `__call__` . . . 9
3.3 Tính Kế thừa (Inheritance) . . . 10
3.4 Quản lý thuộc tính: Siêu tham số (Hyperparameters) và Tham số học được (Learned Parameters) . . . 11

3.5 Thiết kế Custom Estimator chuẩn Scikit-Learn . . . 11
3.6 Mẫu thiết kế Pipeline . . . 12
3.7 Tóm tắt chương . . . 12

4 Tiền xử lý dữ liệu (Data Preprocessing) 13
4.1 Khởi nguồn và vai trò của tiền xử lý dữ liệu . . . 13
4.1.1 "Rác vào, rác ra" . . . 13
4.1.2 Tiền xử lý trong quy trình khai phá tri thức (KDD) . . . 14
4.1.3 Phân loại các tác vụ tiền xử lý . . . 15
4.1.4 Thư viện scikit-learn và giao diện `fit` / `transform` . . . 15
4.2 Các kiểu dữ liệu và lựa chọn phép biến đổi . . . 16
4.3 Mã hóa dữ liệu phân loại . . . 16
4.3.1 Label Encoder . . . 16
4.3.2 One Hot Encoder . . . 18
4.4 Vector hóa văn bản . . . 21
4.4.1 Mô hình túi từ và Count Vectorizer . . . 21
4.4.2 TF-IDF . . . 22
4.5 Chuẩn hóa thang đo đặc trưng (Feature Scaling) . . . 25
4.5.1 Vì sao cần chuẩn hóa thang đo? . . . 25
4.5.2 Min Max Scaler . . . 26
4.5.3 Max Abs Scaler . . . 26
4.5.4 Standard Scaler (chuẩn hóa z-score) . . . 26
4.5.5 Robust Scaler . . . 26
4.5.6 Ví dụ so sánh bốn bộ chuẩn hóa . . . 26
4.6 Kết hợp các bước: Pipeline và ColumnTransformer . . . 28
4.7 Nghiên cứu tình huống: bộ dữ liệu Ames Housing . . . 28
4.7.1 Giới thiệu bộ dữ liệu . . . 28
4.7.2 Những "cái bẫy" khi đọc dữ liệu . . . 29
4.7.3 Đọc và khảo sát dữ liệu . . . 29
4.7.4 Xây dựng quy trình tiền xử lý hoàn chỉnh . . . 30
4.8 Tóm tắt chương . . . 32
4.9 Bài tập . . . 32
Tài liệu tham khảo của chương . . . 33

5 Hồi quy tuyến tính (Linear Regression) 34
5.1 Mô hình Hồi quy tuyến tính đơn biến . . . 34
5.2 Hàm mất mát (Loss Function) . . . 34
5.3 Cài đặt Hồi quy tuyến tính với Scikit-Learn . . . 34
5.4 Tóm tắt chương . . . 35

Chương 1: Giới thiệu về Python

1.1 Python là gì?

Python là một ngôn ngữ lập trình được sử dụng rộng rãi trong các ứng dụng web, phát triển phần mềm, khoa học dữ liệu và máy học (ML). Các nhà phát triển sử dụng Python vì nó hiệu quả, dễ học và có thể chạy trên nhiều nền tảng khác nhau. Phần mềm Python được tải xuống miễn phí, tích hợp tốt với tất cả các loại hệ thống và tăng tốc độ phát triển. (Nguồn: `https://aws.amazon.com/vi/what-is/python/`)

Python hỗ trợ nhiều mô hình lập trình, trong đó có lập trình thủ tục, lập trình hướng đối tượng và lập trình hàm.

Một chương trình Python đơn giản:

```
print("Hello, Python!")
```

Cú pháp của Python có xu hướng sử dụng khoảng trắng để thể hiện cấu trúc của chương trình. Ví dụ:

```
if age >= 18:
    print("Adult")
else:
    print("Under 18")
```

Cách tổ chức này giúp mã nguồn Python có cấu trúc rõ ràng và dễ đọc.

1.2 Lịch sử phát triển của Python

1.2.1 Khởi nguồn & Phiên bản thử nghiệm (1989 – 1991)

Vào cuối năm 1989, khi đang làm việc tại Viện Nghiên cứu Quốc gia về Toán học và Khoa học Máy tính (CWI) ở Hà Lan, Guido van Rossum quyết định viết một ngôn ngữ lập trình mới trong kỳ nghỉ Giáng sinh để làm dự án phụ. Ông muốn tạo ra một ngôn ngữ kết hợp tính đơn giản của ngôn ngữ ABC với khả năng tương tác tốt với hệ điều hành Amoeba.

Tháng 2/1991, Guido đăng tải mã nguồn lên nhóm tin alt.sources với phiên bản Python 0.9.0. Dù chỉ là bản phôi thai, nó đã có những nền móng rất hiện đại: Mô hình OOP, Hệ thống xử lý ngoại lệ, Các kiểu dữ liệu cốt lõi (list, dict, str) và Hệ thống module.

1.2.2 Python 1.0 & Chuẩn hóa quản trị (1994 – 1999)

Tháng 1/1994, Python 1.0 chính thức ra mắt, tích hợp mạnh mẽ các công cụ của Lập trình chức năng (Functional Programming) như lambda, map, filter và reduce.

Năm 1995 (Python 1.2), Guido tiếp tục phát triển Python tại Tập đoàn Sáng kiến Nghiên cứu Quốc gia (CNRI) ở Mỹ, bổ sung khả năng truyền tham số theo tên (Keyword arguments). Trong giai đoạn này, Guido được cộng đồng trao danh hiệu BDFL (Benevolent Dictator for Life).

1.2.3 Python 2.0 & Sự trỗi dậy của Cộng đồng (Tháng 10/2000)

Python 2.0 đánh dấu sự chuyển dịch lớn sang mô hình phát triển mã nguồn mở hoàn toàn. Các tính năng đột phá bao gồm: List Comprehension, Garbage Collector (GC), và ra đời quy trình PEP (Python Enhancement Proposal). Phiên bản Python 2.7 (2010) trở thành phiên bản ổn định nhất của dòng Python 2.

1.2.4 Python 3.0 – ”Cuộc đại tái thiết” (Tháng 12/2008)

Sau nhiều năm phát triển, Python 2 bị tích tụ nhiều lỗi thiết kế. Guido quyết định phát hành Python 3.0 với đặc điểm cốt lõi là Không tương thích ngược (Backwards Incompatible) với Python 2. Những cải tiến quan trọng bao gồm:

- Đồng bộ Unicode cho mọi chuỗi ký tự.
- Thay đổi câu lệnh `print` thành hàm `print()`.
- Tối ưu hóa bộ nhớ bằng cách chuyển các hàm như `range()`, `zip()`, `map()` sang dạng trả về Iterator.

1.2.5 Kỷ nguyên AI/Machine Learning & Khai tử Python 2 (2015 – Nay)

Cú pháp đơn giản giúp Python trở thành lựa chọn hàng đầu trong Khoa học dữ liệu và AI với các thư viện như NumPy, Pandas, Scikit-Learn, TensorFlow, PyTorch.

Ngày 01/01/2020, Python 2 chính thức bị khai tử (EOL), đưa Python 3 trở thành tiêu chuẩn duy nhất. Giai đoạn hiện tại (Python 3.8 – 3.13+) tập trung vào bổ sung các toán tử mới và dự án Faster CPython nhằm tối ưu hóa hiệu năng.

1.3 Guido van Rossum – tác giả của Python

Guido van Rossum là một nhà khoa học máy tính người Hà Lan. Ông là người giữ quyền quyết định tối cao về định hướng kiến trúc của ngôn ngữ (BDFL) cho đến khi từ chức vào năm 2018.

1.4 Tên gọi Python

Tên gọi ”Python” không xuất phát từ loài trăn, mà lấy cảm hứng từ chương trình hài kịch yêu thích của ông trên đài BBC: ”Monty Python’s Flying Circus”.

1.5 Hướng dẫn cài đặt Python và Anaconda

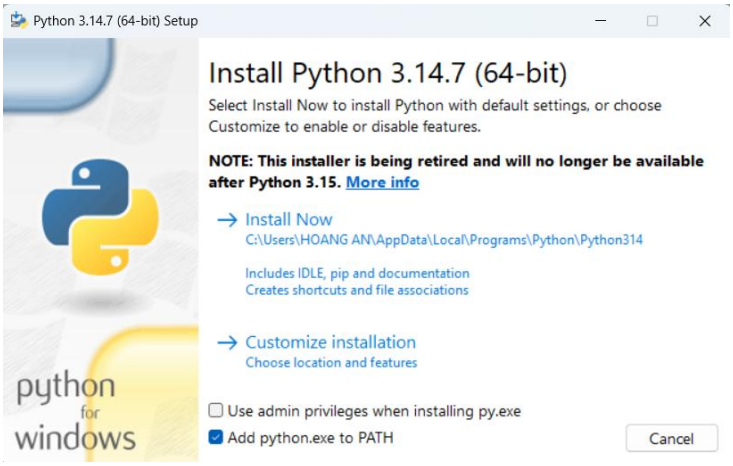
1.5.1 Cài đặt Python

1. Truy cập trang chủ Python: `https://www.python.org/downloads/`



Hình 1.1: Guido van Rossum - Tác giả sáng lập Python. (Nguồn: Sauria Associates, LLC / FlickrVision)

2. Tải phiên bản cài đặt (Ví dụ: Python 3.14.7 cho Windows).
3. Mở file cài đặt. **Lưu ý quan trọng:** Nhớ tick vào ô ”Add python.exe to PATH” trước khi bấm ”Install Now”.
4. Bấm ”Close” khi cài đặt thành công. Có thể chọn ”Disable path length limit” nếu cần.



Hình 1.2: Tick chọn Add python.exe to PATH trong quá trình cài đặt

1.5.2 Cài đặt Anaconda/Miniconda

Anaconda là một nền tảng phân phối Python và R phục vụ cho khoa học dữ liệu.

1. Truy cập trang tải Anaconda: `https://www.anaconda.com/download`
2. Tải bản cài đặt Anaconda Distribution hoặc Miniconda.
3. Mở file cài đặt, chọn ”Just Me”, chọn đường dẫn lưu (ví dụ: `C:\miniconda3`).

4. Trong phần Advanced Installation Options, có thể tick chọn ”Register Miniconda3 as my default Python”.

5. Nhấn ”Install” và đợi quá trình cài đặt hoàn tất.

1.5.3 Tạo môi trường Conda

Sau khi cài đặt Anaconda/Miniconda, mở Anaconda Prompt và sử dụng lệnh sau để tạo một môi trường ảo (ví dụ tên là `ai`) với phiên bản Python 3.14:

```
conda create -n ai python=3.14 anaconda
```

Xác nhận tiến trình cài đặt các gói bằng cách gõ y (yes) khi được yêu cầu.

1.6 Tóm tắt chương

Trong chương này, chúng ta đã tìm hiểu:

- Khái niệm cơ bản về Python và ứng dụng của nó.
- Lịch sử hình thành chi tiết qua từng giai đoạn (từ Python 0.9.0 đến Python 3.x).
- Tiểu sử của tác giả Guido van Rossum và nguồn gốc tên gọi thú vị của Python.
- Các bước cài đặt môi trường lập trình cơ bản với Python và Anaconda.

Chương 2: Cú pháp Python cơ bản

Chương này trình bày các khái niệm nền tảng trong lập trình Python, bao gồm biến, kiểu dữ liệu, toán tử, cấu trúc điều khiển và các hàm toán học. Đây là các kiến thức bắt buộc trước khi đi vào xử lý dữ liệu và Machine Learning.

2.1 Biến và Kiểu dữ liệu

Trong Python, biến được khởi tạo tự động khi gán giá trị. Python là ngôn ngữ *dynamically typed* (định kiểu động).

Định nghĩa 2.1 (Biến trong Python)**.** *Biến là tên gọi đại diện cho một vùng nhớ chứa dữ liệu. Giá trị của biến có thể thay đổi trong quá trình thực thi chương trình.*

2.1.1 Các kiểu dữ liệu cơ bản

Dựa trên nền tảng Python Basic, dưới đây là bảng tổng hợp các kiểu dữ liệu cốt lõi thường xuyên được sử dụng:

Tên	Kiểu dữ liệu	Ví dụ
`str`	Kiểu chuỗi (string)	`a = 'Thien Thanh'`
`int`	Kiểu số nguyên	`a = 79`
`float`	Kiểu số thực	`a = 7.6`
`NoneType`	Kiểu None	`a = None`
`bool`	Kiểu luận lý (True, False)	`a = True`
`list`	Kiểu danh sách	`a = [3, 2, 1, 5]`
`tuple`	Kiểu bộ	`a = (1, 5, 3)`
`dict`	Kiểu từ điển	`b = {'one': 1, 'four': 4}`
`set`	Kiểu tập hợp	`a = {1, 3, 5}`

Bảng 2.1: Các kiểu dữ liệu cơ bản trong Python

Để kiểm tra kiểu dữ liệu của một biến trong Python, ta sử dụng hàm `type()`:

Listing 2.1: Kiểm tra kiểu dữ liệu

```
a = 7
print(type(a))  # Output: <class 'int'>
```

2.2 Toán tử trong Python

Toán tử là thành phần không thể thiếu để thực hiện các phép tính từ cơ bản đến phức tạp trong các thuật toán.

2.2.1 Toán tử cơ bản

Tên toán tử	Ký hiệu	Ví dụ	Kết quả
Cộng	`+`	`3 + 7`	`10`
Trừ	`-`	`7 - 2`	`5`
Nhân	`*`	`2 * 7`	`14`
Chia	`/`	`7 / 5`	`1.4`
Chia lấy phần nguyên	`//`	`10 // 4`	`2`
Chia lấy phần dư	`%`	`9 % 5`	`4`
Lấy mũ	`**`	`2 ** 3`	`8`

Bảng 2.2: Các toán tử số học cơ bản

2.2.2 Toán tử gán mở rộng

Để mã nguồn ngắn gọn hơn, Python hỗ trợ các toán tử gán mở rộng kết hợp phép tính và phép gán:

Tên toán tử	Ký hiệu	Ví dụ	Kết quả
Cộng	`+=`	`a = 7; a += 3`	`10`
Trừ	`-=`	`b = 9; b -= 4`	`5`
Nhân	`*=`	`a = 2; a *= 7`	`14`
Chia	`/=`	`a = 7; b /= 5`	`1.4`
Chia lấy phần nguyên	`//=`	`a = 10; a //= 4`	`2`
Chia lấy phần dư	`%=`	`a = 9; a %= 5`	`4`
Lấy mũ	`**=`	`a = 2; a **= 3`	`8`

Bảng 2.3: Các toán tử gán mở rộng

2.3 Hàm toán học trong Python

Đối với các ứng dụng khoa học dữ liệu, thư viện `math` cung cấp các hằng số và hàm tính toán nâng cao. Để sử dụng, ta cần import thư viện:

Listing 2.2: Import thư viện math

```
1 import math
```

2.3.1 Các hằng số toán học

- Hằng số Euler (E): `math.e` ≈ 2.718
- Hằng số Pi (π): `math.pi` ≈ 3.14159

2.3.2 Các hàm toán học thường dùng

Tên hàm (Toán học)	Mã lệnh Python
$\ln(x)$	`math.log(math.e)`
$\log_2(x)$	`math.log2(3)`
e^x	`math.exp(3)`
$\log_{10}(x)$	`math.log10(3)`
$\sqrt{x}$	`math.sqrt(3)`
a^x	`math.pow(2, 3)`

Bảng 2.4: Một số hàm toán học từ thư viện math

2.4 Cấu trúc dữ liệu nâng cao

Đối với Machine Learning, ngoài mảng 1 chiều, ta thường sử dụng 3 cấu trúc dữ liệu cơ bản bao gồm List, Tuple và Dictionary.

2.4.1 List (Danh sách)

List là tập hợp các phần tử có thứ tự và có thể thay đổi (mutable).

Listing 2.3: Thao tác cơ bản trên List

```
# Khoi tao list
features = [1.2, 3.4, 5.1, 2.8]

# Truy cap va thay doi phan tu
features[0] = 0.9
features.append(4.5)

print("List sau khi cap nhat:", features)
```

2.5 Cấu trúc điều khiển và Vòng lặp

2.5.1 Câu lệnh điều kiện if-else

Listing 2.4: Sử dụng cấu trúc điều kiện

```
score = 0.85

```

```
if score >= 0.8:
    print("Mo hinh dat do chinh xac cao")
else:
    print("Can can thiep hoac toi uu them")
```

2.6 Tóm tắt chương

Trong chương này, chúng ta đã nắm được:

- Cách khai báo biến và chi tiết các kiểu dữ liệu cơ bản trong Python.
- Các toán tử số học và toán tử gán mở rộng.
- Cách sử dụng thư viện `math` cho các hàm toán học phức tạp.
- Cấu trúc dữ liệu nâng cao: List, Tuple, Dictionary.
- Cấu trúc rẽ nhánh trong Python.

Chương 3: Lập trình hướng đối tượng trong Python

Lập trình hướng đối tượng (Object-Oriented Programming - OOP) là phương pháp luận lập trình đóng vai trò then chốt trong việc thiết kế các thư viện Machine Learning hiện đại như Scikit-Learn, PyTorch và TensorFlow. Chương này trình bày các khái niệm OOP cốt lõi và hướng dẫn cách ứng dụng OOP để tự đóng gói một mô hình Machine Learning.

3.1 Lớp và Đối tượng (Class and Object)

Định nghĩa 3.1 (Lớp và Đối tượng)**.** ***Lớp (Class)*** *là khuôn mẫu định nghĩa các thuộc tính (dữ liệu) và phương thức (hành vi).* ***Đối tượng (Object)*** *là một thể hiện cụ thể được khởi tạo từ lớp đó.*

Ví dụ 3.1. *Khai báo một lớp cơ bản lưu trữ mẫu dữ liệu:*

Listing 3.1: Khai báo Class và tạo Object trong Python

```
class DataSample:
    def __init__(self, feature_name, value):
        self.feature_name = feature_name  # Thuoc tinh
        self.value = value

    def display_info(self):               # Phuong thuc
        print(f"Feature: {self.feature_name}, Value: {self.value}")

# Khoi tao doi tuong
sample = DataSample(feature_name="Age", value=25)
sample.display_info()
```

3.2 Các phương thức đặc biệt (Magic Methods)

Các phương thức có cú pháp hai dấu gạch dưới (`__method__`) giúp tùy biến hành vi của đối tượng trong Python.

3.2.1 Phương thức `__init__` và `__call__`

- `__init__`: Hàm khởi tạo (constructor), tự động thực thi khi tạo mới một đối tượng.

- `__call__`: Cho phép gọi trực tiếp một đối tượng giống như một hàm, thường dùng khi thiết kế các Layer trong Neural Network.

Listing 3.2: Sử dụng phương thức __call__

```
class Multiplier:
    def __init__(self, factor):
        self.factor = factor

    def __call__(self, x):
        return x * self.factor

# Su dung nhu mot ham
double = Multiplier(factor=2)
print(double(5))  # Output: 10
```

3.3 Tính Kế thừa (Inheritance)

Tính kế thừa cho phép tạo một lớp mới dựa trên lớp đã có, giúp tái sử dụng mã nguồn và chuẩn hóa giao diện của các bộ biến đổi dữ liệu (Transformers).

Listing 3.3: Kế thừa lớp BaseTransformer trong tiền xử lý dữ liệu

```
class BaseTransformer:
    def fit(self, arr):
        raise NotImplementedError("Phuong thuc fit() bat buoc phai
            duoc trien khai o lop con.")

    def transform(self, arr):
        raise NotImplementedError("Phuong thuc transform() bat buoc
            phai trien khai o lop con.")

    def fit_transform(self, arr):
        self.fit(arr)
        return self.transform(arr)

class MinMaxScaler(BaseTransformer):
    def fit(self, arr):
        self.min_ = min(arr)
        self.max_ = max(arr)
        return self

    def transform(self, arr):
        return [(x - self.min_) / (self.max_ - self.min_) for x in
            arr]

    def inverse_transform(self, arr):
        return [x * (self.max_ - self.min_) + self.min_ for x in arr]
```

3.4 Quản lý thuộc tính: Siêu tham số (Hyperparameters) và Tham số học được (Learned Parameters)

Trong thiết kế Machine Learning, ta cần phân biệt rõ giữa siêu tham số (cấu hình trước khi huấn luyện) và tham số học được (được sinh ra sau khi gọi hàm `fit`).

Listing 3.4: Bắt lỗi an toàn khi quản lý tham số học được

```
class MinMaxScaler(BaseTransformer):
    def __init__(self, feature_range=(0, 1)):
        self.feature_range = feature_range  # Hyperparameter

    def fit(self, arr):
        self.min = float(min(arr))          # Learned Parameter
        self.max = float(max(arr))          # Learned Parameter
        return self

    def transform(self, arr):
        # Kiem tra an toan: Đam bao mo hinh da duoc fit()
        if not hasattr(self, 'min') or not hasattr(self, 'max'):
            raise RuntimeError("Loi: Ban phai goi ham fit() truoc khi
                goi transform()!")

        low, high = self.feature_range
        std = [(x - self.min) / (self.max - self.min) for x in arr]
        return [x * (high - low) + low for x in std]
```

3.5 Thiết kế Custom Estimator chuẩn Scikit-Learn

Trọng tâm ứng dụng OOP trong Machine Learning là đóng gói thuật toán thành một Class có các giao diện chuẩn (`fit`, `predict`, `evaluate`). Dưới đây là triển khai mô hình Hồi quy tuyến tính đơn biến (Simple Linear Regression) bằng công thức đại số.

Listing 3.5: Đóng gói thuật toán Simple Linear Regression

```
class SimpleLinearRegression:
    def __init__(self):
        self.w0 = 0
        self.w1 = 0

    def fit(self, X, y):
        n = len(X)
        mean_x = sum(X) / n
        mean_y = sum(y) / n

        numerator = sum((X[i] - mean_x) * (y[i] - mean_y) for i in
           range(n))
        denominator = sum((X[i] - mean_x) ** 2 for i in range(n))

        if denominator == 0:
```

```
            raise ValueError("Bao loi: Phuong sai cua X bang 0, khong the tinh toan.")

        self.w1 = numerator / denominator
        self.w0 = mean_y - (self.w1 * mean_x)
        return self

    def predict(self, X):
        return [(self.w1 * x_i + self.w0) for x_i in X]

    def evaluate(self, X, y):
        predictions = self.predict(X)
        mse = sum((predictions[i] - y[i]) ** 2 for i in range(len(y))) / len(y)
        return mse
```

3.6 Mẫu thiết kế Pipeline

Pipeline là lớp quản lý chuỗi các bước biến đổi dữ liệu nối tiếp nhau, cho phép tự động hóa toàn bộ quy trình tiền xử lý trước khi đưa vào mô hình dự đoán.

Listing 3.6: Triển khai cấu trúc Pipeline cơ bản

```
class Pipeline:
    def __init__(self, steps):
        self.steps = steps

    def fit_transform(self, arr):
        data = arr
        for transformer in self.steps:
            data = transformer.fit_transform(data)
        return data

    def transform(self, arr):
        data = arr
        for transformer in self.steps:
            data = transformer.transform(data)
        return data
```

3.7 Tóm tắt chương

- Khái niệm Lớp, Đối tượng và tư duy lập trình hướng đối tượng.
- Ứng dụng các Magic Methods (`__init__`, `__call__`) trong xây dựng Framework.
- Nguyên lý Kế thừa giúp tái sử dụng và quản lý mã nguồn linh hoạt.
- Đóng gói thuật toán Machine Learning thành Class theo giao diện tiêu chuẩn.
- Phân biệt rõ ràng siêu tham số và tham số học được thông qua các thuộc tính của đối tượng

Chương 4: Tiền xử lý dữ liệu (Data Preprocessing)

Các thuật toán học máy (Machine Learning – ML) cổ điển như hồi quy tuyến tính, KNN, K-Means, SVM hay PCA đều được xây dựng trên nền tảng đại số tuyến tính và giải tích: chúng nhận đầu vào là **các vector số thực** và thực hiện phép cộng, phép nhân, phép tính khoảng cách giữa các vector đó. Trong khi đó, dữ liệu ngoài đời thực lại rất đa dạng: tên màu sắc, tên khu phố, đoạn văn bản bình luận, mức đánh giá "tốt – trung bình – kém", diện tích nhà tính bằng feet vuông bên cạnh số phòng tắm chỉ từ 0 đến 4… Khoảng cách giữa "dữ liệu thô" và "dữ liệu mà mô hình hiểu được" chính là đối tượng nghiên cứu của chương này.

> *Nguyên tắc cốt lõi:* Mô hình ML không làm việc được trực tiếp trên văn bản hay nhãn chữ — **mọi thứ đều phải được biến đổi thành số**, và phải biến đổi sao cho **không làm sai lệch ý nghĩa** của dữ liệu.

Sau khi học xong chương này, người học có khả năng:

- Trình bày được nguồn gốc lịch sử và vị trí của tiền xử lý dữ liệu trong quy trình khai phá tri thức (KDD) và CRISP-DM.
- Phân biệt các kiểu dữ liệu (định danh, thứ bậc, số, văn bản) và chọn phép biến đổi phù hợp cho từng kiểu.
- Mã hóa dữ liệu phân loại bằng **Label Encoder** và **One Hot Encoder**, hiểu rõ ưu nhược điểm của từng cách.
- Vector hóa văn bản bằng **Count Vectorizer** và **TF-IDF Vectorizer**, tự tính tay được ma trận TF-IDF.
- Chuẩn hóa thang đo đặc trưng bằng **Min Max Scaler**, **Max Abs Scaler**, **Standard Scaler** và **Robust Scaler**.
- Xây dựng quy trình tiền xử lý hoàn chỉnh, tránh rò rỉ dữ liệu, trên bộ dữ liệu thực tế **Ames Housing**.

4.1 Khởi nguồn và vai trò của tiền xử lý dữ liệu

4.1.1 "Rác vào, rác ra"

Ngay từ những ngày đầu của ngành máy tính, giới lập trình viên đã lưu truyền nguyên lý *Garbage In, Garbage Out* (GIGO – "rác vào, rác ra"), thường được gán cho George Fuechsel, một kỹ sư

IBM vào cuối thập niên 1950. Nguyên lý này nói rằng một chương trình dù đúng đến đâu cũng chỉ cho ra kết quả tốt khi dữ liệu đầu vào tốt. Đối với học máy, nguyên lý này còn nghiêm trọng hơn: mô hình *học* trực tiếp từ dữ liệu, nên mọi sai lệch, nhiễu hay cách mã hóa không phù hợp đều được mô hình "học thuộc" và khuếch đại thành dự đoán sai.

4.1.2 Tiền xử lý trong quy trình khai phá tri thức (KDD)

Năm 1996, Fayyad, Piatetsky-Shapiro và Smyth công bố bài báo kinh điển *From Data Mining to Knowledge Discovery in Databases* trên tạp chí *AI Magazine*, chính thức hóa quy trình **khám phá tri thức từ cơ sở dữ liệu** (Knowledge Discovery in Databases – KDD). Trong quy trình này, "khai phá dữ liệu" (tức bước chạy thuật toán) chỉ là **một** bước, đứng sau hàng loạt bước chuẩn bị:

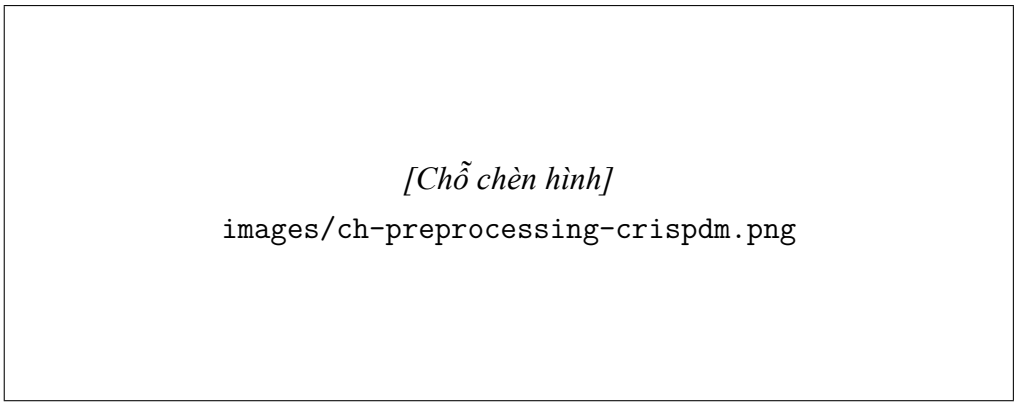
1. **Lựa chọn** (Selection): chọn tập dữ liệu, biến quan tâm.
2. **Tiền xử lý** (Preprocessing): làm sạch, xử lý nhiễu, xử lý giá trị thiếu.
3. **Biến đổi** (Transformation): mã hóa, chuẩn hóa, giảm chiều.
4. **Khai phá dữ liệu** (Data Mining): áp dụng thuật toán.
5. **Diễn giải / Đánh giá** (Interpretation/Evaluation).

[Chỗ chèn hình]

`images/ch-preprocessing-kdd.png`

Hình 4.1: Quy trình KDD theo Fayyad và cộng sự (1996). Tiền xử lý và biến đổi dữ liệu đứng trước bước khai phá dữ liệu.

Cùng thời điểm, một liên minh công nghiệp gồm DaimlerChrysler, SPSS và NCR phát triển chuẩn **CRISP-DM** (Cross-Industry Standard Process for Data Mining, công bố bản 1.0 năm 2000) gồm sáu pha: *Hiểu nghiệp vụ – Hiểu dữ liệu – Chuẩn bị dữ liệu – Mô hình hóa – Đánh giá – Triển khai*. Pha *Chuẩn bị dữ liệu* được nhiều tài liệu thực hành ước tính chiếm phần lớn thời gian của một dự án (con số thường được trích dẫn là khoảng 60–80%). Cuốn *Data Preparation for Data Mining* của Dorian Pyle (1999) là một trong những chuyên khảo đầu tiên dành riêng cho chủ đề này.



Hình 4.2: Sáu pha của CRISP-DM. Pha "Chuẩn bị dữ liệu" là nơi diễn ra phần lớn các kỹ thuật trong chương này.

4.1.3 Phân loại các tác vụ tiền xử lý

Theo Han, Kamber và Pei (*Data Mining: Concepts and Techniques*), tiền xử lý dữ liệu gồm bốn nhóm tác vụ chính:

Nhóm tác vụ	**Nội dung**
Làm sạch dữ liệu	Xử lý giá trị thiếu, dữ liệu nhiễu, ngoại lai, dữ liệu sai định dạng.
Tích hợp dữ liệu	Gộp dữ liệu từ nhiều nguồn, xử lý trùng lặp và mâu thuẫn.
Biến đổi dữ liệu	**Mã hóa** (Label/One Hot Encoder), **vector hóa văn bản** (Count/TF-IDF), **chuẩn hóa thang đo** (các Scaler).
Rút gọn dữ liệu	Giảm số chiều (PCA, SVD – xem chương Giảm chiều dữ liệu), giảm số mẫu.

Bảng 4.1: Bốn nhóm tác vụ tiền xử lý dữ liệu

Chương này tập trung vào nhóm **biến đổi dữ liệu** — nhóm kỹ thuật mà gần như mọi bài toán ML đều phải dùng — và minh họa phần làm sạch dữ liệu qua nghiên cứu tình huống ở cuối chương.

4.1.4 Thư viện scikit-learn và giao diện `fit` / `transform`

Toàn bộ kỹ thuật trong chương được cài đặt bằng thư viện **scikit-learn**. Dự án bắt đầu năm 2007 như một dự án Google Summer of Code của David Cournapeau, sau đó được nhóm nghiên cứu tại INRIA (Pháp) phát triển và phát hành công khai năm 2010; bài báo giới thiệu chính thức của Pedregosa và cộng sự đăng trên *Journal of Machine Learning Research* năm 2011.

Điểm mạnh của scikit-learn là **giao diện thống nhất** (Buitinck và cộng sự, 2013): mọi bộ biến đổi (transformer) đều có ba phương thức:

Phương thức	Ý nghĩa
`fit(X)`	**Học** các tham số từ dữ liệu (từ điển nhãn, min/max, trung bình, độ lệch chuẩn…).
`transform(X)`	**Áp dụng** các tham số đã học để biến đổi dữ liệu.
`fit_transform(X)`	Gộp hai bước trên (chỉ dùng trên tập huấn luyện).
`inverse_transform(X)`	Biến đổi ngược về giá trị gốc (khi phép biến đổi khả nghịch).

Bảng 4.2: Giao diện chung của các bộ biến đổi trong scikit-learn

Định nghĩa 4.1 (Rò rỉ dữ liệu – Data leakage)**.** *Rò rỉ dữ liệu xảy ra khi thông tin từ tập kiểm tra (test) được sử dụng, trực tiếp hay gián tiếp, trong quá trình huấn luyện. Với tiền xử lý, lỗi điển hình là gọi* `fit` *trên* ***toàn bộ*** *dữ liệu trước khi chia train/test. Quy tắc đúng:* ***chỉ*** `fit` ***trên tập huấn luyện****, sau đó dùng* `transform` *cho cả tập huấn luyện và tập kiểm tra.*

4.2 Các kiểu dữ liệu và lựa chọn phép biến đổi

Trước khi chọn kỹ thuật, cần xác định **thang đo** của từng biến. Stevens (1946) phân biệt bốn thang đo: định danh (nominal), thứ bậc (ordinal), khoảng (interval) và tỉ lệ (ratio). Trong thực hành ML, ta gom thành các nhóm sau:

Kiểu dữ liệu	Đặc điểm	Ví dụ	Phép biến đổi
Định danh (nominal)	Các giá trị chỉ khác nhau, **không có thứ tự**	màu sắc, khu phố	One Hot Encoder
Thứ bậc (ordinal)	Có **thứ tự** nhưng khoảng cách không rõ ràng	kém $<$ TB $<$ tốt	Ordinal Encoder
Nhãn đầu ra (target)	Biến cần dự đoán trong phân lớp	“spam”/“ham”	Label Encoder
Số (numeric)	Có giá trị và khoảng cách thực	diện tích, giá	Các Scaler
Văn bản (text)	Chuỗi từ có độ dài thay đổi	bình luận, tài liệu	Count / TF-IDF Vectorizer

Bảng 4.3: Kiểu dữ liệu và kỹ thuật tiền xử lý tương ứng

4.3 Mã hóa dữ liệu phân loại

4.3.1 Label Encoder

Định nghĩa 4.2 (Label Encoding)**.** *Cho tập K nhãn phân biệt $\mathcal{C} = \{c_0, c_1, \ldots, c_{K-1}\}$. Label Encoding là một song ánh*

$$\phi : \mathcal{C} \to \{0, 1, \ldots, K-1\}, \qquad \phi(c_k) = k, \tag{4.1}$$

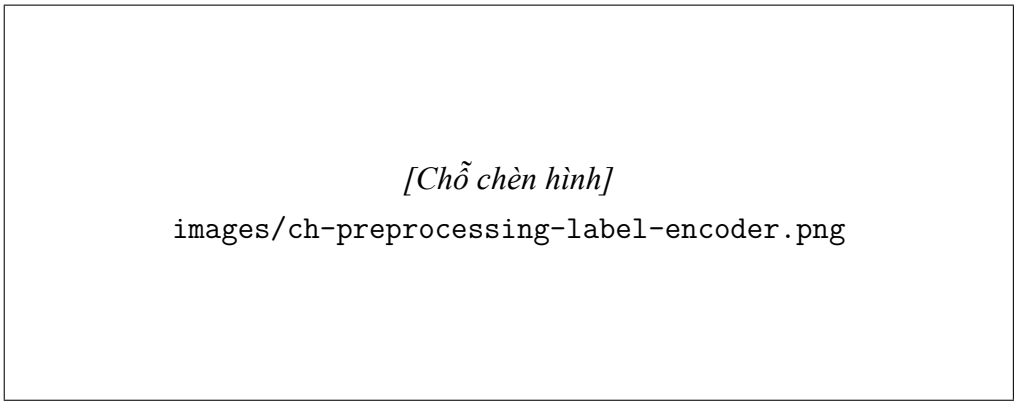
gán cho mỗi nhãn một số nguyên duy nhất. Kết quả là một ***từ điển*** *(dictionary) ánh xạ nhãn $\to$ số.*

Ví dụ 4.1. *Cho dãy đầu vào gồm 7 phần tử:* `red`*,* `green`*,* `blue`*,* `red`*,* `green`*,* `red`*,* `orange`*. Duyệt dãy từ trái sang phải, mỗi khi gặp một nhãn* ***mới*** *ta cấp cho nó số tiếp theo:*

Input	red	green	blue	red	green	red	orange
Output	0	1	2	0	1	0	3

Dictionary	
red	0
green	1
blue	2
orange	3

Bảng 4.4: Label Encoding theo thứ tự xuất hiện



Hình 4.3: Minh họa Label Encoder trên bảng tính: dòng input, dòng output và từ điển ánh xạ.

Để hiểu bản chất thuật toán, ta tự cài đặt Label Encoder chỉ bằng các cấu trúc đã học ở chương Python cơ bản: **list, vòng lặp `for` và dictionary**.

Listing 4.1: Tự cài đặt Label Encoder bằng list, for và dictionary

```
data = ["red", "green", "blue", "red", "green", "red", "orange"]

# Buoc 1: xay dung tu dien nhan -> so (theo thu tu xuat hien)
dictionary = {}
for label in data:
    if label not in dictionary:
        dictionary[label] = len(dictionary)
print(dictionary)
# {'red': 0, 'green': 1, 'blue': 2, 'orange': 3}

# Buoc 2: ma hoa du lieu
encoded = [dictionary[label] for label in data]
print(encoded)          # [0, 1, 2, 0, 1, 0, 3]

# Buoc 3: giai ma (inverse) bang tu dien dao
inverse = {v: k for k, v in dictionary.items()}
print([inverse[v] for v in encoded])
```

Trong scikit-learn, lớp `LabelEncoder` làm việc tương tự nhưng có một điểm khác quan trọng: các nhãn được **sắp xếp theo thứ tự từ điển (alphabet)** trước khi đánh số, chứ không theo thứ tự xuất hiện.

Listing 4.2: Label Encoder với scikit-learn

```
from sklearn.preprocessing import LabelEncoder

```

```
data = ["red", "green", "blue", "red", "green", "red", "orange"]
le = LabelEncoder()
encoded = le.fit_transform(data)

print(le.classes_)     # ['blue' 'green' 'orange' 'red']
print(encoded)         # [3 1 0 3 1 3 2]
print(le.inverse_transform([0, 3]))   # ['blue' 'red']
```

Hạn chế của Label Encoder. Label Encoder tạo ra một **thứ tự giả** giữa các nhãn. Với từ điển ở Bảng 4.4, khoảng cách Euclid giữa các màu là:

$$d(\text{red}, \text{green}) = |0 - 1| = 1, \qquad d(\text{red}, \text{orange}) = |0 - 3| = 3. \tag{4.2}$$

Nghĩa là mô hình sẽ "hiểu" rằng màu đỏ giống màu xanh lá hơn màu cam ba lần — một kết luận hoàn toàn vô nghĩa, chỉ xuất phát từ việc ta đánh số tùy ý. Với các thuật toán dựa trên khoảng cách như **KNN, K-Means** hoặc các mô hình tuyến tính, đây là nguồn sai lệch nghiêm trọng.

> **Lưu ý chuẩn mực:** Trong scikit-learn, `LabelEncoder` được thiết kế để mã hóa **biến mục tiêu** y (nhãn lớp trong bài toán phân lớp). Với các **đặc trưng đầu vào** có thứ bậc, dùng `OrdinalEncoder` (cho phép tự khai báo thứ tự); với đặc trưng định danh, dùng `OneHotEncoder`.

Listing 4.3: Ordinal Encoder với thứ tự do người dùng khai báo

```
from sklearn.preprocessing import OrdinalEncoder

quality = [["Po"], ["TA"], ["Gd"], ["Ex"], ["TA"], ["Fa"]]
oe = OrdinalEncoder(categories=[["Po", "Fa", "TA", "Gd", "Ex"]])
print(oe.fit_transform(quality).ravel())
# [0. 2. 3. 4. 2. 1.]   -> Po < Fa < TA < Gd < Ex
```

4.3.2 One Hot Encoder

Ý tưởng mã hóa "một bit bật" (*one-hot*) xuất phát từ thiết kế mạch số, nơi mỗi trạng thái của máy trạng thái được biểu diễn bằng đúng một flip-flop ở mức 1. Trong thống kê, kỹ thuật tương đương là **biến giả** (dummy variable), được Suits (1957) phổ biến cho hồi quy với biến phân loại.

Định nghĩa 4.3 (One Hot Encoding)**.** *Với tập K nhãn $\mathcal{C} = \{c_0, \ldots, c_{K-1}\}$, One Hot Encoding ánh xạ nhãn c_k thành vector đơn vị thứ k trong $\mathbb{R}^K$:*

$$\psi(c_k) = \mathbf{e}_k = (0, \ldots, 0, \underbrace{1}_{\text{vị trí } k}, 0, \ldots, 0) \in \{0, 1\}^K. \tag{4.3}$$

*Một cột phân loại được thay bằng K cột nhị phân; mỗi hàng có **đúng một** số 1.*

Ưu điểm then chốt: mọi cặp nhãn khác nhau đều **cách đều nhau**:

$$\|\mathbf{e}_i - \mathbf{e}_j\|_2 = \sqrt{2}, \qquad \forall\, i \neq j, \tag{4.4}$$

nên không còn thứ tự giả. Đây là lý do One Hot Encoder là lựa chọn mặc định cho đặc trưng định danh khi dùng **K-Means, KNN**, SVM hay hồi quy tuyến tính.

Ví dụ 4.2. *Cho dãy 9 phần tử (chỉ số 0–8):* `red, blue, red, red, blue, white, red, white, orange`. *Từ điển theo thứ tự xuất hiện là {red: 0, blue: 1, white: 2, orange: 3}, do đó mỗi phần tử trở thành một vector 4 chiều:*

Chỉ số	Input	red (0)	blue (1)	white (2)	orange (3)
0	red	1	0	0	0
1	blue	0	1	0	0
2	red	1	0	0	0
3	red	1	0	0	0
4	blue	0	1	0	0
5	white	0	0	1	0
6	red	1	0	0	0
7	white	0	0	1	0
8	orange	0	0	0	1

Bảng 4.5: Ma trận One Hot (9×4) cho dãy màu sắc

[Chỗ chèn hình]

`images/ch-preprocessing-one-hot-encoder.png`

Hình 4.4: Minh họa One Hot Encoder trên bảng tính: từ điển (Key – Value) và ma trận đầu ra; ô được tô vàng là vị trí số 1 tương ứng với nhãn của từng hàng.

Listing 4.4: Tự cài đặt One Hot Encoder bằng list, for và dictionary

```
data = ["red", "blue", "red", "red", "blue",
        "white", "red", "white", "orange"]

# Buoc 1: tu dien (Key -> Value)
dictionary = {}
for label in data:
    if label not in dictionary:
        dictionary[label] = len(dictionary)
K = len(dictionary)          # so cot = so nhan phan biet = 4

# Buoc 2: moi phan tu -> vector co dung mot so 1
one_hot = []
for label in data:
    row = [0] * K
    row[dictionary[label]] = 1
```

```
    one_hot.append(row)

for i, row in enumerate(one_hot):
    print(i, data[i], row)
# 0 red    [1, 0, 0, 0]
# 1 blue   [0, 1, 0, 0]
# ...
# 8 orange [0, 0, 0, 1]
```

Listing 4.5: One Hot Encoder với scikit-learn và pandas

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import OneHotEncoder

data = np.array(["red", "blue", "red", "red", "blue",
                 "white", "red", "white", "orange"]).reshape(-1, 1)

ohe = OneHotEncoder(handle_unknown="ignore")
X = ohe.fit_transform(data)       # ket qua la ma tran thua (sparse)
print(ohe.get_feature_names_out())
# ['x0_blue' 'x0_orange' 'x0_red' 'x0_white']   (sap xep alphabet)
print(X.toarray()[:3])
# [[0. 0. 1. 0.]
#  [1. 0. 0. 0.]
#  [0. 0. 1. 0.]]

# Nhan moi chua tung gap -> vector toan 0 thay vi bao loi
print(ohe.transform([["green"]]).toarray())   # [[0. 0. 0. 0.]]

# Cach nhanh voi pandas
print(pd.get_dummies(pd.Series(data.ravel()), dtype=int).head(3))
```

Một số vấn đề cần lưu ý.

- **Bẫy biến giả** (dummy variable trap): K cột one-hot luôn có tổng bằng 1, tức phụ thuộc tuyến tính hoàn toàn với cột hệ số chặn. Với hồi quy tuyến tính không chính quy hóa, nên bỏ một cột bằng `OneHotEncoder(drop="first")` hoặc `pd.get_dummies(..., drop_first=True)`.
- **Bùng nổ số chiều**: biến có hàng nghìn giá trị (mã bưu chính, mã sản phẩm) sinh ra hàng nghìn cột. Khi đó cần gộp các nhãn hiếm, dùng ma trận thưa hoặc các kỹ thuật như target encoding, hashing.
- **Nhãn chưa từng gặp** ở tập kiểm tra: dùng `handle_unknown="ignore"` để tránh lỗi khi triển khai.

Tiêu chí	Label / Ordinal Encoder	One Hot Encoder
Số cột sinh ra	1	K (số nhãn)
Tạo thứ tự giả	Có	Không
Phù hợp với	Biến mục tiêu; đặc trưng thứ bậc; mô hình cây	Đặc trưng định danh; KNN, K-Means, SVM, mô hình tuyến tính
Nhược điểm	Sai lệch khoảng cách	Tăng số chiều, ma trận thưa

Bảng 4.6: So sánh Label Encoder và One Hot Encoder

4.4 Vector hóa văn bản

4.4.1 Mô hình túi từ và Count Vectorizer

Ý tưởng biểu diễn một văn bản bằng **tần suất các từ** có từ rất sớm: nhà ngôn ngữ học Zellig Harris (1954) trong *Distributional Structure* đã nhắc đến "túi từ" (bag of words), và Hans Peter Luhn (1957) tại IBM đề xuất dùng tần suất từ để đánh chỉ mục tài liệu tự động. Năm 1975, Salton, Wong và Yang hệ thống hóa thành **mô hình không gian vector** (Vector Space Model): mỗi văn bản là một vector trong không gian có số chiều bằng kích thước từ vựng.

Định nghĩa 4.4 (Count Vectorizer)**.** *Cho kho văn bản (corpus) $D = \{d_1, \ldots, d_N\}$ và từ vựng $V = \{t_1, \ldots, t_M\}$ gồm mọi từ xuất hiện trong D. Count Vectorizer biểu diễn văn bản d bằng vector*

$$\mathbf{x}_d = \big(\mathrm{tf}(t_1, d), \mathrm{tf}(t_2, d), \ldots, \mathrm{tf}(t_M, d)\big), \tag{4.5}$$

trong đó $\mathrm{tf}(t, d)$ *(term frequency) là số lần từ t xuất hiện trong d. Cả kho văn bản trở thành ma trận văn bản – từ (document-term matrix) kích thước $N \times M$.*

Ví dụ 4.3. *Xét kho văn bản gồm 3 câu:*

- d_1*: "the cats are in the house"*
- d_2*: "the dogs are in the house and outside"*
- d_3*: "the cats and dogs are friends"*

Từ vựng (sắp xếp alphabet) gồm $M = 9$ từ: and, are, cats, dogs, friends, house, in, outside, the.

	and	are	cats	dogs	friends	house	in	outside	the
d_1	0	1	1	0	0	1	1	0	2
d_2	1	1	0	1	0	1	1	1	2
d_3	1	1	1	1	1	0	0	0	1
$\mathrm{df}(t)$	2	3	2	2	1	2	2	1	3

Bảng 4.7: Ma trận đếm từ (Count Vectorizer) và tần suất văn bản $\mathrm{df}(t)$

Listing 4.6: Count Vectorizer với scikit-learn

```
1 from sklearn.feature_extraction.text import CountVectorizer
2
```

```
corpus = [
    "the cats are in the house",
    "the dogs are in the house and outside",
    "the cats and dogs are friends",
]
cv = CountVectorizer()
X = cv.fit_transform(corpus)

print(cv.vocabulary_)
# {'the': 8, 'cats': 2, 'are': 1, 'in': 6, 'house': 5,
#  'dogs': 3, 'and': 0, 'outside': 7, 'friends': 4}
print(X.toarray())
# [[0 1 1 0 0 1 1 0 2]
#  [1 1 0 1 0 1 1 1 2]
#  [1 1 1 1 1 0 0 0 1]]
```

Mặc định `CountVectorizer` chuyển chữ về chữ thường và tách từ bằng biểu thức chính quy `(?u)\b\w\w+\b`, tức **bỏ qua các từ chỉ có 1 ký tự**. Các tham số hay dùng: `stop_words` (loại từ dừng), `ngram_range=(1,2)` (lấy thêm cặp từ), `min_df`/`max_df` (loại từ quá hiếm/quá phổ biến).

Chuẩn hóa L2. Văn bản dài có số đếm lớn hơn chỉ vì dài hơn. Để so sánh công bằng, mỗi vector được chia cho chuẩn Euclid của nó:

$$\hat{\mathbf{x}}_d = \frac{\mathbf{x}_d}{\|\mathbf{x}_d\|_2}, \qquad \|\mathbf{x}_d\|_2 = \sqrt{\textstyle\sum_j x_{dj}^2}. \tag{4.6}$$

Với d_1: $\|\mathbf{x}_{d_1}\|_2^2 = 1+1+1+1+2^2 = 8$, nên các số 1 trở thành $1/\sqrt{8} \approx 0{,}3536$ và số 2 trở thành $2/\sqrt{8} \approx 0{,}7071$. Tương tự $\|\mathbf{x}_{d_2}\|_2 = \sqrt{10}$ và $\|\mathbf{x}_{d_3}\|_2 = \sqrt{6}$:

	and	are	cats	dogs	friends	house	in	outside	the
d_1	0	0.3536	0.3536	0	0	0.3536	0.3536	0	0.7071
d_2	0.3162	0.3162	0	0.3162	0	0.3162	0.3162	0.3162	0.6325
d_3	0.4082	0.4082	0.4082	0.4082	0.4082	0	0	0	0.4082

Bảng 4.8: Ma trận đếm sau khi chuẩn hóa L2 theo từng hàng

4.4.2 TF-IDF

Count Vectorizer có một nhược điểm: những từ xuất hiện ở **mọi** văn bản (như "the", "are") có số đếm cao nhưng gần như không giúp phân biệt văn bản. Năm 1972, Karen Spärck Jones đề xuất trong bài báo *A statistical interpretation of term specificity and its application in retrieval* (*Journal of Documentation*) rằng trọng số của một từ nên tỉ lệ nghịch với số văn bản chứa nó — đó là **tần suất văn bản nghịch đảo** (Inverse Document Frequency – IDF). Salton và Buckley (1988) sau đó tổng kết các biến thể trọng số TF-IDF, trở thành chuẩn mực của ngành truy hồi thông tin suốt nhiều thập kỷ.

Định nghĩa 4.5 (TF-IDF)**.** *Với N là số văn bản và* $\mathrm{df}(t)$ *là số văn bản chứa từ t, trọng số TF-IDF của từ t trong văn bản d là*

$$\textit{tf-idf}(t,d) = \mathrm{tf}(t,d) \times \mathrm{idf}(t). \tag{4.7}$$

Dạng kinh điển của IDF là $\text{idf}(t) = \ln \dfrac{N}{\text{df}(t)}$. *Thư viện scikit-learn dùng hai biến thể có cộng thêm 1 để từ xuất hiện ở mọi văn bản không bị triệt tiêu hoàn toàn:*

$$\textit{Không làm trơn (}\texttt{smooth_idf=False}\textit{)}: \quad \text{idf}(t) = 1 + \ln \frac{N}{\text{df}(t)}, \tag{4.8}$$

$$\textit{Làm trơn (}\texttt{smooth_idf=True}\textit{, mặc định)}: \quad \text{idf}(t) = 1 + \ln \frac{N+1}{\text{df}(t)+1}. \tag{4.9}$$

Công thức làm trơn tương đương việc thêm một "văn bản ảo" chứa mọi từ đúng một lần, giúp tránh phép chia cho 0.

Ví dụ 4.4 (Tính tay IDF)**.** *Với* $N = 3$ *và* df *ở Bảng 4.7, chỉ có ba giá trị* $\text{df} \in \{1, 2, 3\}$*:*

$\text{df}(t)$	**Các từ**	**IDF không làm trơn (4.8)**	**IDF làm trơn (4.9)**
1	friends, outside	$1 + \ln 3 = 2{,}0986$	$1 + \ln 2 = 1{,}6931$
2	and, cats, dogs, house, in	$1 + \ln 1{,}5 = 1{,}4055$	$1 + \ln \frac{4}{3} = 1{,}2877$
3	are, the	$1 + \ln 1 = 1$	$1 + \ln 1 = 1$

Bảng 4.9: Giá trị IDF cho kho 3 văn bản

Nhân từng cột của ma trận đếm (Bảng 4.7) với IDF tương ứng, ta được ma trận TF-IDF (chưa chuẩn hóa). Chẳng hạn với từ "cats" trong d_1: $\text{tf} = 1$, $\text{df} = 2$ nên $\text{tf-idf} = 1 \times 1{,}2877 = 1{,}2877$ (làm trơn); với "the" trong d_1: $2 \times 1 = 2$.

	and	are	cats	dogs	friends	house	in	outside	the
IDF làm trơn (`smooth_idf=True`*)*									
d_1	0	1	1.2877	0	0	1.2877	1.2877	0	2
d_2	1.2877	1	0	1.2877	0	1.2877	1.2877	1.6931	2
d_3	1.2877	1	1.2877	1.2877	1.6931	0	0	0	1
IDF không làm trơn (`smooth_idf=False`*)*									
d_1	0	1	1.4055	0	0	1.4055	1.4055	0	2
d_2	1.4055	1	0	1.4055	0	1.4055	1.4055	2.0986	2
d_3	1.4055	1	1.4055	1.4055	2.0986	0	0	0	1

Bảng 4.10: Ma trận TF-IDF chưa chuẩn hóa (`norm=None`)

[Chỗ chèn hình]

`images/ch-preprocessing-tfidf-excel.png`

Hình 4.5: Tính TF-IDF trên bảng tính: ma trận đếm, $\text{df}(t)$, $\text{idf}(t, D)$ làm trơn, ma trận TF-IDF và ma trận chuẩn hóa L2.

[Chỗ chèn hình]

`images/ch-preprocessing-tfidf-nosmooth.png`

Hình 4.6: Ma trận TF-IDF với IDF không làm trơn (giá trị 1,40547 và 2,09861) — khớp với kết quả `smooth_idf=False` của scikit-learn.

Chuẩn hóa L2 và độ tương đồng cosine. Mặc định `TfidfVectorizer` còn chuẩn hóa L2 từng hàng. Áp dụng cho ma trận làm trơn ở Bảng 4.10:

	and	are	cats	dogs	friends	house	in	outside	the
d_1	0	0.3166	0.4077	0	0	0.4077	0.4077	0	0.6333
d_2	0.3382	0.2626	0	0.3382	0	0.3382	0.3382	0.4447	0.5252
d_3	0.4105	0.3188	0.4105	0.4105	0.5397	0	0	0	0.3188

Bảng 4.11: Ma trận TF-IDF mặc định của scikit-learn (`smooth_idf=True`, `norm="l2"`)

Vì mỗi hàng có độ dài 1, tích vô hướng giữa hai hàng chính là **độ tương đồng cosine**:

$$\cos(d_i, d_j) = \hat{\mathbf{x}}_{d_i} \cdot \hat{\mathbf{x}}_{d_j}. \tag{4.10}$$

Ta có $\cos(d_1, d_2) \approx 0{,}6915$, $\cos(d_1, d_3) \approx 0{,}4702$, $\cos(d_2, d_3) \approx 0{,}5288$: hai câu cùng nói về "in the house" (d_1, d_2) giống nhau nhất — phù hợp với trực giác.

Listing 4.7: TF-IDF Vectorizer với các cấu hình khác nhau

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

corpus = [
    "the cats are in the house",
    "the dogs are in the house and outside",
    "the cats and dogs are friends",
]

# 1) Chi dem tu (tuong duong CountVectorizer)
v = TfidfVectorizer(use_idf=False, smooth_idf=False, norm=None)
print(v.fit_transform(corpus).toarray())

# 2) TF-IDF khong lam tron, chua chuan hoa -> 1.4055, 2.0986
v = TfidfVectorizer(use_idf=True, smooth_idf=False, norm=None)
print(v.fit_transform(corpus).toarray().round(4))

# 3) TF-IDF lam tron, chua chuan hoa -> 1.2877, 1.6931
```

```
v = TfidfVectorizer(use_idf=True, smooth_idf=True, norm=None)
print(v.fit_transform(corpus).toarray().round(4))
print(v.idf_.round(4))
# [1.2877 1.     1.2877 1.2877 1.6931 1.2877 1.2877 1.6931 1.    ]

# 4) Cau hinh mac dinh: lam tron + chuan hoa L2
v = TfidfVectorizer()
X = v.fit_transform(corpus)
print(X.toarray().round(4))
print(cosine_similarity(X).round(4))
# [[1.     0.6915 0.4702]
#  [0.6915 1.     0.5288]
#  [0.4702 0.5288 1.    ]]
```

Cấu hình	`use_idf`	`smooth_idf`	`norm`
Đếm từ thuần túy	False	–	None
Đếm từ chuẩn hóa L2	False	–	`"l2"`
TF-IDF thô	True	False	None
TF-IDF mặc định	True	True	`"l2"`

Bảng 4.12: Các cấu hình của `TfidfVectorizer` dùng trong chương

4.5 Chuẩn hóa thang đo đặc trưng (Feature Scaling)

4.5.1 Vì sao cần chuẩn hóa thang đo?

Xét hai đặc trưng của một căn nhà: diện tích lô đất (hàng nghìn đến hàng trăm nghìn feet vuông) và số phòng tắm (0 đến 4). Trong khoảng cách Euclid

$$d(\mathbf{x}, \mathbf{z}) = \sqrt{(x_1 - z_1)^2 + (x_2 - z_2)^2}, \tag{4.11}$$

chênh lệch diện tích sẽ áp đảo hoàn toàn chênh lệch số phòng tắm, dù trên thực tế số phòng tắm có thể quan trọng không kém. Chuẩn hóa thang đo đưa mọi đặc trưng về cùng một "đơn vị" để:

- Các thuật toán dựa trên **khoảng cách** (KNN, K-Means, SVM với nhân RBF) đối xử công bằng giữa các đặc trưng.
- **Gradient Descent** hội tụ nhanh hơn, vì mặt hàm mất mát bớt bị kéo dài theo một chiều.
- **PCA** không bị chi phối bởi đặc trưng có phương sai lớn chỉ do đơn vị đo.
- Các mô hình có **chính quy hóa** (Ridge, Lasso) phạt các hệ số một cách công bằng.

Ngược lại, các mô hình dựa trên **cây quyết định** (Decision Tree, Random Forest, XGBoost) chỉ so sánh ngưỡng trên từng đặc trưng nên **không** cần chuẩn hóa.

Ký hiệu một cột đặc trưng là $x = (x_1, \ldots, x_n)$. Bốn bộ chuẩn hóa phổ biến trong scikit-learn được định nghĩa như sau.

4.5.2 Min Max Scaler

$$x_i' = \frac{x_i - \min(x)}{\max(x) - \min(x)} \in [0, 1]. \tag{4.12}$$

Tổng quát hơn, với `feature_range=(a, b)`: $x_i'' = a + x_i'(b - a)$. Phép biến đổi giữ nguyên hình dạng phân phối, nhưng **rất nhạy với ngoại lai** vì min và max chính là các giá trị cực trị.

4.5.3 Max Abs Scaler

$$x_i' = \frac{x_i}{\max_j |x_j|} \in [-1, 1]. \tag{4.13}$$

Max Abs Scaler không dịch chuyển dữ liệu (không trừ đi giá trị nào), nên **giữ nguyên các số 0**. Đây là lựa chọn phù hợp cho **ma trận thưa** như ma trận TF-IDF ở mục trước: trừ đi trung bình sẽ biến mọi số 0 thành khác 0 và phá vỡ tính thưa.

4.5.4 Standard Scaler (chuẩn hóa z-score)

$$z_i = \frac{x_i - \mu}{\sigma}, \qquad \mu = \frac{1}{n}\sum_{i=1}^{n} x_i, \qquad \sigma = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(x_i - \mu)^2}. \tag{4.14}$$

Sau biến đổi, cột có trung bình 0 và độ lệch chuẩn 1. Lưu ý scikit-learn dùng độ lệch chuẩn **tổng thể** (chia cho n, tức `ddof=0`), khác với `pandas.Series.std()` mặc định chia cho $n - 1$. Standard Scaler là lựa chọn mặc định cho hồi quy tuyến tính, hồi quy logistic, SVM, PCA và mạng nơ-ron.

4.5.5 Robust Scaler

$$x_i' = \frac{x_i - \text{median}(x)}{\text{IQR}(x)}, \qquad \text{IQR} = Q_3 - Q_1, \tag{4.15}$$

trong đó Q_1, Q_3 là các phân vị 25% và 75%. Trung vị và khoảng tứ phân vị **gần như không bị ảnh hưởng bởi ngoại lai**, nên Robust Scaler phù hợp với dữ liệu lệch hoặc có giá trị bất thường.

4.5.6 Ví dụ so sánh bốn bộ chuẩn hóa

Ví dụ 4.5. *Cho cột $x = (2, 4, 6, 8, 30)$, trong đó 30 là một ngoại lai. Các đại lượng cần thiết:* $\min = 2$, $\max = 30$, $\max|x| = 30$, $\mu = 10$, $\sigma = \sqrt{104} \approx 10{,}198$, $\text{median} = 6$, $Q_1 = 4$, $Q_3 = 8$, $\text{IQR} = 4$.

x	MinMax	MaxAbs	Standard	Robust
2	0.0000	0.0667	-0.7845	-1.0
4	0.0714	0.1333	-0.5883	-0.5
6	0.1429	0.2000	-0.3922	0.0
8	0.2143	0.2667	-0.1961	0.5
30	1.0000	1.0000	1.9612	6.0

Bảng 4.13: Kết quả của bốn bộ chuẩn hóa trên cột có ngoại lai

Quan sát Bảng 4.13: với Min Max Scaler, bốn giá trị "bình thường" bị ép vào đoạn $[0;\ 0{,}21]$ — gần như mất khả năng phân biệt — chỉ vì một ngoại lai. Robust Scaler giữ các giá trị bình thường trải đều trong $[-1;\ 0{,}5]$ và đẩy ngoại lai ra xa (6,0), giúp ngoại lai **dễ nhận diện** thay vì làm méo phần còn lại.

Listing 4.8: So sánh bốn bộ chuẩn hóa trong scikit-learn

```
import numpy as np
from sklearn.preprocessing import (MinMaxScaler, MaxAbsScaler,
                                   StandardScaler, RobustScaler)

x = np.array([2, 4, 6, 8, 30], dtype=float).reshape(-1, 1)

for scaler in [MinMaxScaler(), MaxAbsScaler(),
               StandardScaler(), RobustScaler()]:
    print(type(scaler).__name__, scaler.fit_transform(x).ravel().
        round(4))
# MinMaxScaler   [0.     0.0714 0.1429 0.2143 1.    ]
# MaxAbsScaler   [0.0667 0.1333 0.2    0.2667 1.    ]
# StandardScaler [-0.7845 -0.5883 -0.3922 -0.1961  1.9612]
# RobustScaler   [-1.  -0.5  0.   0.5  6. ]
```

[Chỗ chèn hình]

images/ch-preprocessing-scalers.png

Hình 4.7: Phân phối của một đặc trưng trước và sau khi áp dụng bốn bộ chuẩn hóa (gợi ý: vẽ histogram hoặc boxplot bằng matplotlib).

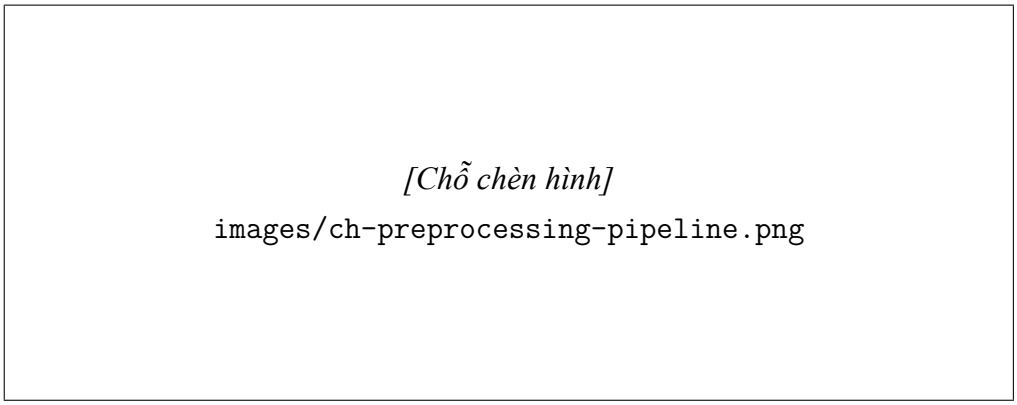
Bộ chuẩn hóa	**Miền giá trị**	**Bền với ngoại lai**	**Khi nào dùng**
MinMaxScaler	$[0, 1]$	Không	Cần miền cố định (ảnh, mạng nơ-ron), dữ liệu ít ngoại lai
MaxAbsScaler	$[-1, 1]$	Không	Ma trận thưa (TF-IDF), dữ liệu đã tâm ở 0
StandardScaler	Không giới hạn	Trung bình	Mặc định cho mô hình tuyến tính, SVM, PCA
RobustScaler	Không giới hạn	Có	Dữ liệu lệch, nhiều ngoại lai

Bảng 4.14: Tóm tắt lựa chọn bộ chuẩn hóa

4.6 Kết hợp các bước: Pipeline và ColumnTransformer

Dữ liệu thực tế thường có **nhiều kiểu cột cùng lúc**. scikit-learn cung cấp:

- `ColumnTransformer`: áp dụng các bộ biến đổi khác nhau cho các nhóm cột khác nhau rồi ghép kết quả.
- `Pipeline`: xâu chuỗi các bước tiền xử lý và mô hình thành một đối tượng duy nhất. Khi gọi `pipeline.fit(X_train, y_train)`, mọi bước `fit` chỉ nhìn thấy tập huấn luyện — đây là cách tự động **tránh rò rỉ dữ liệu**.



Hình 4.8: Sơ đồ ColumnTransformer: cột số đi qua SimpleImputer + Scaler, cột định danh qua One Hot Encoder, cột thứ bậc qua Ordinal Encoder, sau đó ghép lại thành ma trận đặc trưng.

4.7 Nghiên cứu tình huống: bộ dữ liệu Ames Housing

4.7.1 Giới thiệu bộ dữ liệu

Bộ dữ liệu **Ames Housing** do Dean De Cock (Đại học Truman State) công bố năm 2011 trên *Journal of Statistics Education*, như một lựa chọn thay thế hiện đại cho bộ dữ liệu giá nhà Boston kinh điển. Dữ liệu ghi nhận các giao dịch bán nhà ở thành phố Ames, bang Iowa (Hoa Kỳ) trong giai đoạn 2006–2010, lấy từ Văn phòng Định giá Tài sản của thành phố.

File `data/AmesHousing.txt` (phân tách bằng ký tự tab) gồm **2930 dòng** và **82 cột**: hai cột định danh bản ghi (`Order`, `PID`), biến mục tiêu `SalePrice` (giá bán, USD) và 79 biến mô tả căn nhà. Đây là bộ dữ liệu lý tưởng cho chương này vì nó chứa **đủ mọi kiểu dữ liệu** đã học:

Kiểu	Ví dụ cột	Giá trị mẫu
Định danh	`MS Zoning`, `Neighborhood`, `Street`, `Bldg Type`	RL, RM, FV; NAmes, Gilbert; Pave, Grvl
Định danh mã hóa bằng số	`MS SubClass`	020, 060, 120 (mã loại nhà, **không phải** đại lượng)
Thứ bậc	`Exter Qual`, `Kitchen Qual`, `Bsmt Qual`	Po < Fa < TA < Gd < Ex
Số rời rạc	`Full Bath`, `Garage Cars`, `Yr Sold`	0–4; 0–5; 2006–2010
Số liên tục	`Lot Area`, `Gr Liv Area`, `SalePrice`	`Lot Area` tối đa 215245 ft^2

Bảng 4.15: Các kiểu dữ liệu trong Ames Housing

4.7.2 Những "cái bẫy" khi đọc dữ liệu

Quan sát kỹ file dữ liệu cho thấy nhiều vấn đề rất điển hình mà người làm ML phải xử lý:

1. **"NA" không phải lúc nào cũng là thiếu dữ liệu.** Ở các cột `Alley`, `Fence`, `Pool QC`, `Fireplace Qu`, `Garage Type`, `Bsmt Qual`…, chuỗi `NA` mang nghĩa *"không có"* (không có hẻm, không có hàng rào, không có garage). Nếu đọc bằng `pd.read_csv` mặc định, pandas sẽ biến tất cả `NA` thành `NaN` và ta **đánh mất thông tin**. Trong khi đó, ô **rỗng** (ví dụ `Lot Frontage` ở nhiều dòng, `Mas Vnr Type` ở một số dòng) mới thực sự là dữ liệu thiếu.

2. **Khoảng trắng thừa:** cột `Sale Type` có giá trị `"WD "` (có dấu cách cuối). Không làm sạch thì `"WD"` và `"WD "` bị coi là hai nhãn khác nhau khi One Hot.

3. **Số nhưng là phân loại:** `MS SubClass` (020, 060…) là mã loại nhà; dùng trực tiếp như số sẽ gây thứ tự giả giống hệt vấn đề của Label Encoder.

4. **Lỗi nhập liệu:** có bản ghi với `Garage Yr Blt` = 2207 — một năm không thể xảy ra (nhiều khả năng là 2007).

5. **Ngoại lai:** một vài căn có `Gr Liv Area` trên 4000 ft^2 nhưng giá bán thấp bất thường (giao dịch bán từng phần). Chính De Cock khuyến nghị loại các căn nhà trên 4000 ft^2 khi dùng dữ liệu cho giảng dạy. Cột `Lot Area` cũng lệch phải rất mạnh (tối đa 215245 ft^2).

4.7.3 Đọc và khảo sát dữ liệu

Listing 4.9: Đọc Ames Housing và giữ nguyên ý nghĩa của "NA"

```
import pandas as pd

# keep_default_na=False: KHONG bien "NA" thanh NaN
# na_values=[""]       : chi o rong moi la du lieu thieu
df = pd.read_csv("data/AmesHousing.txt", sep="\t",
                 keep_default_na=False, na_values=[""])
print(df.shape)                          # (2930, 82)

```

```
# Lam sach khoang trang thua o cac cot chu
obj_cols = df.select_dtypes("object").columns
df[obj_cols] = df[obj_cols].apply(lambda s: s.str.strip())

# MS SubClass la ma phan loai -> doi sang kieu chuoi
df["MS SubClass"] = df["MS SubClass"].astype(str)

# Thong ke du lieu thieu that su
missing = df.isna().sum()
print(missing[missing > 0].sort_values(ascending=False))

# Sua loi nhap lieu va loai ngoai lai theo khuyen nghi cua De Cock
df.loc[df["Garage Yr Blt"] > 2010, "Garage Yr Blt"] = None
df = df[df["Gr Liv Area"] <= 4000]
print(df.shape)

print(df["Exter Qual"].value_counts())
print(df[["Lot Area", "Gr Liv Area", "SalePrice"]].describe())
```

[Chỗ chèn hình]

images/ch-preprocessing-ames-outliers.png

Hình 4.9: Biểu đồ phân tán `Gr Liv Area` – `SalePrice` của Ames Housing; các điểm bên phải ngưỡng 4000 ft^2 là ngoại lai.

4.7.4 Xây dựng quy trình tiền xử lý hoàn chỉnh

Ta chọn một tập con đặc trưng đại diện cho từng kiểu dữ liệu và áp dụng đúng kỹ thuật đã học cho từng nhóm:

Listing 4.10: Pipeline tiền xử lý cho Ames Housing

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import (OneHotEncoder, OrdinalEncoder,
                                   StandardScaler, RobustScaler)

nominal = ["MS Zoning", "Neighborhood", "Bldg Type", "MS SubClass",
           "Alley", "Garage Type"]
```

```
quality = ["Exter Qual", "Kitchen Qual", "Bsmt Qual", "Fireplace Qu"]
numeric = ["Gr Liv Area", "Total Bsmt SF", "Garage Cars",
           "Full Bath", "Year Built", "Overall Qual"]
skewed  = ["Lot Area", "Lot Frontage"]

# "NA" o cac cot chat luong = khong co -> muc thap nhat
levels = ["NA", "Po", "Fa", "TA", "Gd", "Ex"]

X = df[nominal + quality + numeric + skewed]
y = np.log(df["SalePrice"])      # log giup giam do lech cua gia

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

preprocess = ColumnTransformer([
    ("nom", Pipeline([
        ("impute", SimpleImputer(strategy="most_frequent")),
        ("onehot", OneHotEncoder(handle_unknown="ignore")),
    ]), nominal),
    ("ord", Pipeline([
        ("impute", SimpleImputer(strategy="constant", fill_value="NA"
           )),
        ("ordinal", OrdinalEncoder(categories=[levels] * len(quality)
           )),
    ]), quality),
    ("num", Pipeline([
        ("impute", SimpleImputer(strategy="median")),
        ("scale", StandardScaler()),
    ]), numeric),
    ("skew", Pipeline([
        ("impute", SimpleImputer(strategy="median")),
        ("scale", RobustScaler()),
    ]), skewed),
])

# CHI fit tren tap huan luyen, transform cho ca hai tap
X_train_t = preprocess.fit_transform(X_train)
X_test_t  = preprocess.transform(X_test)
print(X_train_t.shape, X_test_t.shape)
print(preprocess.get_feature_names_out()[:10])
```

Ma trận `X_train_t` thu được chỉ gồm các số thực, đã đồng nhất thang đo và không còn giá trị thiếu — sẵn sàng cho các mô hình hồi quy ở chương tiếp theo. Khi ghép `preprocess` với một mô hình trong cùng một `Pipeline`, toàn bộ quy trình có thể được đánh giá bằng kiểm định chéo mà không lo rò rỉ dữ liệu:

Listing 4.11: Ghép tiền xử lý với mô hình trong một Pipeline

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import cross_val_score

model = Pipeline([("prep", preprocess), ("reg", LinearRegression())])
scores = cross_val_score(model, X_train, y_train, cv=5, scoring="r2")
```

```
print(scores.mean().round(3))
```

4.8 Tóm tắt chương

- Tiền xử lý dữ liệu là bước nền tảng trong quy trình KDD (Fayyad và cộng sự, 1996) và pha Chuẩn bị dữ liệu của CRISP-DM; nguyên lý "rác vào, rác ra" nhắc rằng chất lượng mô hình bị giới hạn bởi chất lượng dữ liệu.
- Mô hình ML chỉ làm việc với số: dữ liệu chữ phải được mã hóa sao cho **không tạo ra quan hệ giả**.
- **Label Encoder** ánh xạ nhãn $\rightarrow$ số nguyên, gọn nhẹ nhưng tạo thứ tự giả; dùng cho biến mục tiêu. **Ordinal Encoder** dùng cho đặc trưng thứ bậc.
- **One Hot Encoder** biến mỗi nhãn thành một vector đơn vị, mọi nhãn cách đều $\sqrt{2}$; phù hợp với KNN, K-Means, mô hình tuyến tính.
- **Count Vectorizer** biểu diễn văn bản bằng số lần xuất hiện của từ; **TF-IDF** (Spärck Jones, 1972) giảm trọng số của các từ phổ biến; chuẩn hóa L2 cho phép đo độ tương đồng cosine bằng tích vô hướng.
- Bốn bộ chuẩn hóa **MinMax, MaxAbs, Standard, Robust** khác nhau về miền giá trị, khả năng giữ tính thưa và độ bền với ngoại lai.
- Luôn `fit` trên tập huấn luyện; dùng `Pipeline` và `ColumnTransformer` để quy trình gọn gàng và tránh rò rỉ dữ liệu.

4.9 Bài tập

1. Cho dãy gồm các phần tử `cat`, `dog`, `fish`, `dog`, `cat`, `bird`. Hãy (a) tự cài đặt Label Encoder theo thứ tự xuất hiện; (b) so sánh với kết quả của `LabelEncoder` trong scikit-learn và giải thích vì sao khác nhau.
2. Với dãy ở bài 1, tự cài đặt One Hot Encoder bằng list, for và dictionary. Chứng minh rằng khoảng cách Euclid giữa hai vector one-hot khác nhau bất kỳ luôn bằng $\sqrt{2}$.
3. Thêm câu d_4 = "the birds are outside" vào kho văn bản ở mục TF-IDF. Tính tay $\mathrm{df}(t)$ và $\mathrm{idf}(t)$ (làm trơn) cho các từ *are, outside, the, birds*, sau đó kiểm tra bằng `TfidfVectorizer`.
4. Dùng bảng tính (Excel/Google Sheets) dựng lại toàn bộ Bảng 4.7–4.11. Gợi ý: hàm `LN`, `SQRT`, `SUMSQ`.
5. Cho cột $x = (10, 12, 11, 13, 12, 100)$. Tính tay kết quả của bốn bộ chuẩn hóa và nhận xét bộ nào phù hợp nhất. Vì sao?
6. Giải thích vì sao gọi `StandardScaler().fit(X)` trên toàn bộ dữ liệu trước khi chia train/test là rò rỉ dữ liệu. Thiết kế thí nghiệm trên Ames Housing để minh họa.
7. Với Ames Housing: đếm số lượng giá trị `NA` ("không có") và ô rỗng (thiếu thật sự) trong từng cột; bổ sung các cột `Heating QC`, `Garage Finish`, `Foundation` vào pipeline với kỹ thuật mã hóa phù hợp.

Tài liệu tham khảo của chương

1. Buitinck, L. et al. (2013). API design for machine learning software: experiences from the scikit-learn project. *ECML PKDD Workshop: Languages for Data Mining and Machine Learning*.

2. De Cock, D. (2011). Ames, Iowa: Alternative to the Boston Housing Data as an End of Semester Regression Project. *Journal of Statistics Education*, 19(3).

3. Fayyad, U., Piatetsky-Shapiro, G., & Smyth, P. (1996). From Data Mining to Knowledge Discovery in Databases. *AI Magazine*, 17(3), 37–54.

4. Han, J., Kamber, M., & Pei, J. (2011). *Data Mining: Concepts and Techniques* (3rd ed.), Chương 3: Data Preprocessing. Morgan Kaufmann.

5. Harris, Z. S. (1954). Distributional Structure. *Word*, 10(2–3), 146–162.

6. Luhn, H. P. (1957). A Statistical Approach to Mechanized Encoding and Searching of Literary Information. *IBM Journal of Research and Development*, 1(4), 309–317.

7. Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

8. Pyle, D. (1999). *Data Preparation for Data Mining*. Morgan Kaufmann.

9. Salton, G., Wong, A., & Yang, C. S. (1975). A Vector Space Model for Automatic Indexing. *Communications of the ACM*, 18(11), 613–620.

10. Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5), 513–523.

11. Spärck Jones, K. (1972). A statistical interpretation of term specificity and its application in retrieval. *Journal of Documentation*, 28(1), 11–21.

12. Stevens, S. S. (1946). On the Theory of Scales of Measurement. *Science*, 103(2684), 677–680.

13. Suits, D. B. (1957). Use of Dummy Variables in Regression Equations. *Journal of the American Statistical Association*, 52(280), 548–551.

14. Wirth, R., & Hipp, J. (2000). CRISP-DM: Towards a Standard Process Model for Data Mining. *Proceedings of the 4th International Conference on the Practical Applications of Knowledge Discovery and Data Mining*.

Chương 5: Hồi quy tuyến tính (Linear Regression)

Hồi quy tuyến tính là một trong những thuật toán học có giám sát (Supervised Learning) cơ bản và phổ biến nhất trong Machine Learning, dùng để dự đoán giá trị đầu ra liên tục.

5.1 Mô hình Hồi quy tuyến tính đơn biến

Định nghĩa 5.1 (Hồi quy tuyến tính đơn biến)**.** *Cho tập dữ liệu* $(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)$. *Mô hình biểu diễn mối quan hệ giữa biến độc lập* x *và biến phụ thuộc* y *theo dạng tuyến tính:*

$$y = w_0 + w_1 x + \epsilon \tag{5.1}$$

trong đó w_0 *là hệ số chặn (intercept),* w_1 *là hệ số góc (slope), và* ϵ *là sai số ngẫu nhiên.*

5.2 Hàm mất mát (Loss Function)

Để tìm các hệ số w_0, w_1 tối ưu, người ta sử dụng sai số bình phương trung bình (Mean Squared Error - MSE):

$$J(w_0, w_1) = \frac{1}{2n} \sum_{i=1}^{n} (y_i - (w_0 + w_1 x_i))^2 \tag{5.2}$$

Mục tiêu của bài toán là tìm (w_0, w_1) sao cho $J(w_0, w_1)$ đạt giá trị nhỏ nhất.

5.3 Cài đặt Hồi quy tuyến tính với Scikit-Learn

Dưới đây là đoạn code Python minh họa việc xây dựng mô hình Linear Regression trên tập dữ liệu thực tế:

Listing 5.1: Thực thi Linear Regression bằng Scikit-Learn

```
import numpy as np
from sklearn.linear_model import LinearRegression

# 1. Tao du lieu gia dinh
X = np.array([[1], [2], [3], [4], [5]])
y = np.array([2.1, 3.9, 6.1, 8.2, 9.9])

```

```
# 2. Khoi tao va huan luyen mo hinh
model = LinearRegression()
model.fit(X, y)

# 3. In ket qua trong so w0 va w1
print("He so w1 (slope):", model.coef_[0])
print("He so w0 (intercept):", model.intercept_)

# 4. Du doan gia tri moi
y_pred = model.predict([[6]])
print("Du doan cho X=6:", y_pred[0])
```

5.4 Tóm tắt chương

- Bản chất mô hình Hồi quy tuyến tính và bài toán tối ưu hàm MSE.
- Phương pháp hạ độ dốc (Gradient Descent) để cập nhật trọng số.
- Thao tác huấn luyện mô hình bằng thư viện `scikit-learn`.